

```

import static org.junit.Assert.assertEquals;
import static org.junit.Assert.assertFalse;
import static org.junit.Assert.assertTrue;

import org.junit.Test;

import components.naturalnumber.NaturalNumber;

/**
 * JUnit test fixture for {@code NaturalNumber}'s constructors and kernel
 * methods.
 *
 * @author cl_c2231am03 (Tyler zhuangzhuang He & Meng Zhang)
 */
public abstract class NaturalNumberTest {

    /**
     * Invokes the appropriate {@code NaturalNumber} constructor for the
     * implementation under test and returns the result.
     *
     * @return the new number
     * @ensures constructorTest = 0
     */
    protected abstract NaturalNumber constructorTest();

    /**
     * Invokes the appropriate {@code NaturalNumber} constructor for the
     * implementation under test and returns the result.
     *
     * @param i
     *        {@code int} to initialize from
     * @return the new number
     * @requires i >= 0
     * @ensures constructorTest = i
     */
    protected abstract NaturalNumber constructorTest(int i);

    /**
     * Invokes the appropriate {@code NaturalNumber} constructor for the
     * implementation under test and returns the result.
     *
     * @param s
     *        {@code String} to initialize from
     * @return the new number
     * @requires there exists n: NATURAL (s = TO_STRING(n))
     * @ensures s = TO_STRING(constructorTest)
     */
    protected abstract NaturalNumber constructorTest(String s);

    /**
     * Invokes the appropriate {@code NaturalNumber} constructor for the
     * implementation under test and returns the result.
     *
     * @param n
     *        {@code NaturalNumber} to initialize from
     * @return the new number
     * @ensures constructorTest = n
     */
    protected abstract NaturalNumber constructorTest(NaturalNumber n);

    /**
     * Invokes the appropriate {@code NaturalNumber} constructor for the
     * reference implementation and returns the result.
     *
     * @return the new number
     * @ensures constructorRef = 0
     */
    protected abstract NaturalNumber constructorRef();

```

```

/**
 * Invokes the appropriate {@code NaturalNumber} constructor for the
 * reference implementation and returns the result.
 *
 * @param i
 *      {@code int} to initialize from
 * @return the new number
 * @requires i >= 0
 * @ensures constructorRef = i
 */
protected abstract NaturalNumber constructorRef(int i);

/**
 * Invokes the appropriate {@code NaturalNumber} constructor for the
 * reference implementation and returns the result.
 *
 * @param s
 *      {@code String} to initialize from
 * @return the new number
 * @requires there exists n: NATURAL (s = TO_STRING(n))
 * @ensures s = TO_STRING(constructorRef)
 */
protected abstract NaturalNumber constructorRef(String s);

/**
 * Invokes the appropriate {@code NaturalNumber} constructor for the
 * reference implementation and returns the result.
 *
 * @param n
 *      {@code NaturalNumber} to initialize from
 * @return the new number
 * @ensures constructorRef = n
 */
protected abstract NaturalNumber constructorRef(NaturalNumber n);

// test cases for four constructors: default, int, String, NaturalNumber

// test cases for constructor default
@Test
public void testConstructor() {
    NaturalNumber n = this.constructorTest();
    NaturalNumber expected = this.constructorRef();

    assertEquals(expected, n);
}

// test cases for constructor integer: 0, nonzero, int max
// case 1: 0
@Test
public void testConstructorIntZero() {
    NaturalNumber n = this.constructorTest(0);
    NaturalNumber expected = this.constructorRef(0);

    assertEquals(expected, n);
}

// case 2: non zero
@Test
public void testConstructorIntNonZero() {
    NaturalNumber n = this.constructorTest(100);
    NaturalNumber expected = this.constructorRef(100);

    assertEquals(expected, n);
}

// case 3: integer max
@Test
public void testConstructorIntMax() {

```

```
NaturalNumber n = this.constructorTest(Integer.MAX_VALUE);
NaturalNumber expected = this.constructorRef(Integer.MAX_VALUE);

    assertEquals(expected, n);
}

// test cases for constructor String: "0", regular, > Integer.MAX_VALUE
// case 1: "0"
@Test
public void testConstructorStringZero() {
    NaturalNumber n = this.constructorTest("0");
    NaturalNumber expected = this.constructorRef("0");

    assertEquals(expected, n);
}

// case 2: "123"(regular)
@Test
public void testConstructorStringDigits() {
    NaturalNumber n = this.constructorTest("123");
    NaturalNumber expected = this.constructorRef("123");

    assertEquals(expected, n);
}

// case 3: Integer.MAX_VALUE(2147483647) + 300
@Test
public void testConstructorStringOverIntMax() {
    String greaterThanIntMax = "2147483947";
    NaturalNumber n = this.constructorTest(greaterThanIntMax);
    NaturalNumber expected = this.constructorRef(greaterThanIntMax);

    assertEquals(expected, n);
}

// case 4: much larger than int max
@Test
public void testConstructorStringLargeNumber() {
    String largeNumber = "123456789012345678901234567890";
    NaturalNumber n = this.constructorTest(largeNumber);
    NaturalNumber expected = this.constructorRef(largeNumber);

    assertEquals(expected, n);
}

// test cases for constructor NaturalNumber: 0, nonzero
// case 1: 0
@Test
public void testConstructorNaturalNumberZero() {
    NaturalNumber original = this.constructorRef(0);
    NaturalNumber n = this.constructorTest(original);
    NaturalNumber expected = this.constructorRef(0);

    assertEquals(expected, n);
}

// case 2: non zero
@Test
public void testConstructorNaturalNumberNonZero() {
    NaturalNumber original = this.constructorRef(105);
    NaturalNumber n = this.constructorTest(original);
    NaturalNumber expected = this.constructorRef(105);

    assertEquals(expected, n);
}

// case 3: aliasing
@Test
public void testConstructorNaturalNumberAliasing() {
```

```
NaturalNumber original = this.constructorRef(105);
NaturalNumber n = this.constructorTest(original);
NaturalNumber expected = this.constructorRef(105);

original.multiplyBy10(9);

assertEquals(expected, n);
}

// case 4: constructor does not change original NaturalNumber
@Test
public void testConstructorNaturalNumberPreservesArgument() {
    NaturalNumber original = this.constructorRef("98765432109876543210");
    NaturalNumber originalExpected = this
        .constructorRef("98765432109876543210");
    NaturalNumber n = this.constructorTest(original);
    NaturalNumber expected = this.constructorRef("98765432109876543210");

    assertEquals(expected, n);
    assertEquals(originalExpected, original);
}

// test cases for multiplyBy10
// case 1: rep = 0, k = 3(non zero)    rep.multiplyBy10(k)
@Test
public void testZeroMultiplyBy10NonZero() {
    NaturalNumber n = this.constructorTest(0);
    NaturalNumber expected = this.constructorRef(3);

    n.multiplyBy10(3);

    assertEquals(expected, n);
}

// case 2: rep = 120, k = 3(non zero)    rep.multiplyBy10(k)
@Test
public void testNonZeroMultiplyBy10NonZero() {
    NaturalNumber n = this.constructorTest(120);
    NaturalNumber expected = this.constructorRef(1203);

    n.multiplyBy10(3);

    assertEquals(expected, n);
}

// case 3: rep = 120, k = 0    rep.multiplyBy10(k)
@Test
public void testMultiplyBy10Zero() {
    NaturalNumber n = this.constructorTest(120);
    NaturalNumber expected = this.constructorRef(1200);

    n.multiplyBy10(0);

    assertEquals(expected, n);
}

// case 4: rep = 0, k = 0    rep.multiplyBy10(k)
@Test
public void testZeroMultiplyBy10Zero() {
    NaturalNumber n = this.constructorTest(0);
    NaturalNumber expected = this.constructorRef(0);

    n.multiplyBy10(0);

    assertEquals(expected, n);
}

// case 5: rep = 120, k = 9    rep.multiplyBy10(k)
@Test
```

```
public void testMultiplyBy10Nine() {
    NaturalNumber n = this.constructorTest(120);
    NaturalNumber expected = this.constructorRef(1209);

    n.multiplyBy10(9);

    assertEquals(expected, n);
}

// case 6: large number, k = 7
@Test
public void testLargeNumberMultiplyBy10Seven() {
    NaturalNumber n = this.constructorTest("98765432109876543210");
    NaturalNumber expected = this.constructorRef("987654321098765432107");

    n.multiplyBy10(7);

    assertEquals(expected, n);
}

// test cases for divideBy10: 0, one/multiple digit, ending multiple 0s
// case 1: rep = 0
@Test
public void testDivideBy10Zero() {
    NaturalNumber n = this.constructorTest(0);
    NaturalNumber expected = this.constructorRef(0);

    int remainder = n.divideBy10();

    assertEquals(0, remainder);
    assertEquals(expected, n);
}

// case 2: rep = 5 (one digit)
@Test
public void testDivideBy10OneDigit() {
    NaturalNumber n = this.constructorTest(5);
    NaturalNumber expected = this.constructorRef(0);

    int remainder = n.divideBy10();

    assertEquals(5, remainder);
    assertEquals(expected, n);
}

// case 3: rep = 12345 (more than one digit)
@Test
public void testDivideBy10MultipleDigits() {
    NaturalNumber n = this.constructorTest(12345);
    NaturalNumber expected = this.constructorRef(1234);

    int remainder = n.divideBy10();

    assertEquals(5, remainder);
    assertEquals(expected, n);
}

// case 4: rep = 1000 (ending more than one zero)
@Test
public void testDivideBy10MultipleZeros() {
    NaturalNumber n = this.constructorTest(1000);
    NaturalNumber expected = this.constructorRef(100);

    int remainder = n.divideBy10();

    assertEquals(0, remainder);
    assertEquals(expected, n);
}
```

```
// case 5: large number ending in nonzero digit
@Test
public void testDivideBy10LargeNumberNonZeroRemainder() {
    NaturalNumber n = this.constructorTest("1234567890123456789");
    NaturalNumber expected = this.constructorRef("123456789012345678");

    int remainder = n.divideBy10();

    assertEquals(9, remainder);
    assertEquals(expected, n);
}

// case 6: large number ending in 0
@Test
public void testDivideBy10LargeNumberZeroRemainder() {
    NaturalNumber n = this.constructorTest("12345678901234567890");
    NaturalNumber expected = this.constructorRef("1234567890123456789");

    int remainder = n.divideBy10();

    assertEquals(0, remainder);
    assertEquals(expected, n);
}

// test cases for isZero: true, false
@Test
public void testIsZeroTrue() {
    NaturalNumber n = this.constructorTest(0);

    assertTrue(n.isZero());
}

@Test
public void testIsZeroFalse() {
    NaturalNumber n = this.constructorTest(8);

    assertFalse(n.isZero());
}
}
```